Muhammad Ghufran Luqman ID: 250204162

OOP Typing Race Simulator Part 1 Report

Task 1:

How encapsulation safeguards data

Encapsulation stops data from being accessed directly to prevent unintended changes and unauthorized modifications to data. By setting all the variables to private, the variables can only be accessed from within (getter and setter) methods inside the class – meaning that they can only be modified in the intended way set by the developer.

The accessor methods are:

- getAccuracy
- getProgress
- getName
- getSymbol
- getBurnoutTurnsRemaining
- isBurntOut

The mutator methods are:

- burnOut
- recoverFromBurnOut
- resetToStart
- typeCharacter
- slideBack
- setAccuracy
- setSymbol

slideBack validation logic:

> If the calculated new progress is below 0 (calculated by doing progress – given number of characters), then it sets the progress to 0. A progress of below zero should not be possible because the player should not be able to go further back than the starting line.
>
> Otherwise, it is not negative, so it sets the progress equal to the new progress

setAccuracy validation logic:

> If the given new accuracy is below 0 then it sets the player's accuracy to 0.
>
> If the given new accuracy is above 1 then it sets the player's accuracy to 1.
>
> If none of those conditions are met, then it sets the player's accuracy to the given new accuracy.
>
> The accuracy must be between 0 and 1 because an accuracy of 0 represents 0% and an accuracy of 1 represents 100% – it should not be possible to have an accuracy rating outside of these bounds.

These methods ensure that the data can only be placed within appropriate boundaries – and encapsulation ensures that the data can only be accessed through these methods, thus overall ensuring that the data cannot fall outside the intended boundaries.

Testing Evidence:

Before each test major and separate, I run the reset command to reset the state so that each test is independent. I also defined methods such as printTestCase and lineBreak so that the testing is clearer. printTestCase takes in the method I am running, and the expected result from that method, and lastly the actual result. It then checks if the expected result equals the actual result and prints whether or not the test was successful. lineBreak prints the given number of dashes.

Below are the helper methods I defined:

```java
public static<T> void printTestCase(String methodName, T expected, T actual) {
    System.out.println(methodName + ":\nExpected: " + expected + "\nActual: " + actual);
    testIfSuccess(expected, actual);
}

public static void lineBreak(int numberOfDashes) {
    System.out.print("\n");
    for (int i=0;i<numberOfDashes;i++) {
        System.out.print("-");
    }
    System.out.println("\n");
}

public static<T> void testIfSuccess(T a, T b) {
    if (a.equals(b)) {
        System.out.println("Success");
    }
    else {
        System.out.println("Failure");
    }
}
```

Testing Normal Forward Movement (typeCharacter())

```
public static void main(String[] a) {
    Typist playerTest = new Typist(typistSymbol: 't', typistName: "test player", typistAccuracy: 0.3);

    playerTest.resetToStart();

    System.out.println("Testing that normal forward movement is possible via typeCharacter():");
    System.out.println("Starting progress: " + playerTest.getProgress());
    int numberOfCharacters = 3;
    for (int i=1;i<=numberOfCharacters;i++) { // type 3 characters
        playerTest.typeCharacter();
    }

    printTestCase(methodName: "getProgress()", numberOfCharacters, playerTest.getProgress());
```

After running the `typeCharacter()` command three times, the progress went up to 3 (from 0).

```
Testing that normal forward movement is possible via typeCharacter():
Starting progress: 0
getProgress():
Expected: 3
Actual: 3
Success
```

This proves the method works because the starting progress was 0, then the `typeCharacter()` method was called 3 times, and the progress after was 3.

Testing resetToStart()

I ran `typeCharacter()` once as well as `burnOut(1)` to make the progress and burnout turns not zero before reset (to check that the reset actually works):

```java
playerTest.resetToStart();

System.out.println("Testing that resetToStart() clears both progress and burnout state:");
playerTest.typeCharacter();

System.out.println("Current progress (before reset): "+playerTest.getProgress());
playerTest.burnOut(turns: 1);
System.out.println("isBurntOut (before reset): "+playerTest.isBurntOut());
System.out.println("getBurnoutTurnsRemaining (before reset): "+playerTest.getBurnoutTurnsRemaining());

// ----------
lineBreak(numberOfDashes: 10);

playerTest.resetToStart();

printTestCase(methodName: "getProgress() (after reset)", expected: 0, playerTest.getProgress());

// ----------
lineBreak(numberOfDashes: 10);

printTestCase(methodName: "isBurntOut() (after reset)", expected: false, playerTest.isBurntOut());

// ----------
lineBreak(numberOfDashes: 10);
printTestCase(methodName: "getBurnoutTurnsRemaining()", expected: 0, playerTest.getBurnoutTurnsRemaining());
```

```
Testing that resetToStart() clears both progress and burnout state:
Current progress (before reset): 1
isBurntOut (before reset): true
getBurnoutTurnsRemaining (before reset): 1

----------

getProgress() (after reset):
Expected: 0
Actual: 0
Success

----------

isBurntOut() (after reset):
Expected: false
Actual: false
Success

----------

getBurnoutTurnsRemaining():
Expected: 0
Actual: 0
Success
```

This works because the progress was not 0, and the burnout turns remaining was not 0 either. Moreover, the typist was in a state of being burnt out. After running the `resetToStart()` command, the progress, burnout turns remaining, and burnout state were all reset successfully, showing that the method works as intended.

Testing burnOut() and recoverFromBurnout()

Calling `burnOut(2)` and `recoverFromBurnout()`:

```
playerTest.resetToStart();

System.out.println("Testing if burnout correctly counts down turn by turn:");
playerTest.burnOut(turns: 2);
playerTest.recoverFromBurnout();

printTestCase(methodName: "isBurntOut()", expected: true, playerTest.isBurntOut());

// ----------
lineBreak(numberOfDashes: 10);

printTestCase(methodName: "getBurnoutTurnsRemaining()", expected: 1, playerTest.getBurnoutTurnsRemaining());

// ----------
lineBreak(numberOfDashes: 10);

System.out.println("Testing burnout clears at 0:\n");
playerTest.recoverFromBurnout();

printTestCase(methodName: "isBurntOut()", expected: false, playerTest.isBurntOut());

// ----------
lineBreak(numberOfDashes: 10);

printTestCase(methodName: "getBurnoutTurnsRemaining()", expected: 0, playerTest.getBurnoutTurnsRemaining());
```

```
Testing if burnout correctly counts down turn by turn:
isBurntOut():
Expected: true
Actual: true
Success

----------

getBurnoutTurnsRemaining():
Expected: 1
Actual: 1
Success
```

This works because there were 2 burnout turns, but after running the `recoverFromBurnout()` command, there was 1 turn left.

Continuing from the last test where the number of burnout turns remaining is 1, I ran this command `recoverFromBurnout()` again, to reset the number of burnout terms to 0.

```
Testing burnout clears at 0:

isBurntOut():
Expected: false
Actual: false
Success

---------

getBurnoutTurnsRemaining():
Expected: 0
Actual: 0
Success
```

This shows that burnout correctly counts down turn by turn.

Testing slideBack()

Calling `slideBack(5)`, the program only reset the progress to 0 as expected.

```
playerTest.resetToStart();

numberOfCharacters = 3;
for (int i=1;i<=numberOfCharacters;i++) { // type 3 characters
    playerTest.typeCharacter();
}

System.out.println("Testing if progress can go below zero after calling slideBack():");
playerTest.slideBack(amount: 5);
printTestCase(methodName: "getProgress()", expected: 0, playerTest.getProgress());
```

```
Testing if progress can go below zero after calling slideBack():
getProgress():
Expected: 0
Actual: 0
Success
```

This works because, after typing 3 characters, the progress is 3 (proven to work due to when I tested typeCharacter() in a previous test), but after calling `slideBack(5)`, the program resets the progress to 0.

Testing setAccuracy()

I ran `setAccuracy(-0.3)` before testing the first case, since -0.3 is below 0.0, and then ran `setAccuracy(1.7)` before testing the second case since 1.7 is above 1.0

```
playerTest.resetToStart();

System.out.println("Testing that accuracy cannot be set outside the 0.0-1.0 range:");
playerTest.setAccuracy(-0.3);

printTestCase(methodName: "getAccuracy()", expected: 0.0, playerTest.getAccuracy());

// ----------
lineBreak(numberOfDashes: 10);
playerTest.setAccuracy(newAccuracy: 1.7);
printTestCase(methodName: "getAccuracy()", expected: 1.0, playerTest.getAccuracy());
```

```
Testing that accuracy cannot be set outside the 0.0-1.0 range:
getAccuracy():
Expected: 0.0
Actual: 0.0
Success

----------

getAccuracy():
Expected: 1.0
Actual: 1.0
Success
```

The program kept the accuracy within the bounds as expected, therefore this test was also successful.

Task 2:

Errors:

```
// Accuracy thresholds for mistype and burnout events
// (Ty tuned these values "by feel". They may need adjustment.)
private static final double MISTYPE_BASE_CHANCE = 0.3;
private static final int    SLIDE_BACK_AMOUNT   = 2;
private static final int    BURNOUT_DURATION    = 3;
```

The mistype base chance seems to be okay. In the screenshot from the specification, it seemed like “QWERTY_QUEEN” slided back 2 spaces after mistyping so I will keep the slide back amount to 2. Furthermore, I decreased the burnout duration to 2 since in the screenshot from the specification, “HUNT_N_PECK” is burnt out for 2 turns.

My code:

```
// Accuracy thresholds for mistype and burnout events
// (Ty tuned these values "by feel". They may need adjustment.)
private static final double MISTYPE_BASE_CHANCE = 0.3;
private static final int    SLIDE_BACK_AMOUNT   = 2;
private static final int    BURNOUT_DURATION     = 2;
```

```
// Reset all typists to the start of the passage
// (Ty was in a hurry here)
seat1Typist.resetToStart();
seat2Typist.resetToStart();
```

He didn’t reset seat3Typist.

My code:

```
// Reset all typists to the start of the passage
// (Ty was in a hurry here)
seat1Typist.resetToStart();
seat2Typist.resetToStart();
seat3Typist.resetToStart();
```

```
    // TODO (Task 2a): Print the winner's name here
```

He didn’t print the winner’s name.

My code:

```
// TODO (Task 2a): Print the winner's name here
Typist[] players = new Typist[3];
players[0] = seat1Typist;
players[1] = seat2Typist;
players[2] = seat3Typist;

for (Typist t : players) {
    if (raceFinishedBy(t)) {
        System.out.println(t.getName());
    }
}
```

I put all the typists into an array, and cycle through them. Then, for each one I check if they have finished the race. If they have, I print their name. Since this code only runs when the race is finished, and since there can only be one winner, this code will print exactly one winner when run.

```
// Mistype check — the probability should reflect the typist's accuracy
if (Math.random() < theTypist.getAccuracy() * MISTYPE_BASE_CHANCE)
{
    theTypist.slideBack(SLIDE_BACK_AMOUNT);
}
```

Math.random, theTypist.getAccuracy() and MISTYPE_BASE_CHANCE are all numbers between 0 and 1.

This is not the correct logic. If we say (theTypist.getAccuracy()*MISTYPE_BASE_CHANCE) = a, then we can say the operation is:

If (Math.random() < a) then a mistype happens.

Abstracting the specific syntax, we can say a = accuracy * base_chance.

On the operation (Math.random() < a), if take some examples we can see the relationship if a is higher or lower:

e.g. a = 0.288 and Math.random() = 0.5, then it's (0.5<0.288).

e.g. a=0.24 and Math.random() = 0.5, then it's (0.5<0.240).

When a is bigger, (Math.random() < a) is more likely to be true. This is because there are more numbers between 0 and 0.288 than there are numbers between 0 and 0.240, therefore when a is bigger, (Math.random() < a) is more likely to be true. When that is true, a mistype happens. Therefore we want that to be more likely to be true when the accuracy is lower.

However, if we look at how a is calculated, it is the opposite:

a = accuracy*base_chance.

If we take base_chance as 0.3 (since I have left it as that for now), and take these examples:

Accuracy = 0.96, then a = 0.96 * 0.3 = 0.288

Accuracy = 0.8, then a = 0.8 * 0.3 = 0.240

When the accuracy was higher, a was higher. When a is higher, the chance of a mistype happening is higher. Therefore, when accuracy is higher, with our current logic, the chance of a mistype is higher. But we want the opposite, if the accuracy is higher, then the chance of a mistype should be lower.

This can be changed if we change accuracy to (1-accuracy).

Testing these out with example values:

Accuracy = 0.96 therefore (1-accuracy) = 0.04, so 0.04*0.3=0.012

Accuracy = 0.8 therefore (1-accuracy) = 0.2, so 0.2*0.3=0.060

Therefore, when accuracy is higher, a is lower, so mistype chance is lower.

But, the chance of a random number between 0 and 1 being lower than 0.06 is still pretty unlikely. Therefore we need to change the base chance.

Testing out when base chance = 0.8:

(1-0.96)*0.8 = 0.032

(1-0.8)*0.8 = 0.360

These are reasonable values in my opinion, therefore I will also chance the mistype base chance to 0.8.

My code:

```
// Accuracy thresholds for mistype and burnout events
// (Ty tuned these values "by feel". They may need adjustment.)
private static final double MISTYPE_BASE_CHANCE = 0.8;
private static final int    SLIDE_BACK_AMOUNT   = 2;
private static final int    BURNOUT_DURATION     = 2;
```

(MISTYPE_BASE_CHANCE was previously 0.3)

```
// Mistype check – the probability should reflect the typist's accuracy
if (Math.random() < (1-theTypist.getAccuracy()) * MISTYPE_BASE_CHANCE)
{
    theTypist.slideBack(SLIDE_BACK_AMOUNT);
}
```

```
153         /**
154          * Returns true if the given typist has completed the full passage.
155          *
156          * @param theTypist the typist to check
157          * @return true if their progress has reached or passed the passage length
158          */
159         private boolean raceFinishedBy(Typist theTypist)
160         {
161             // Ty was confident this condition was correct
162             if (theTypist.getProgress() == passageLength)
163             {
164                 return true;
165             }
166             else
167             {
168                 return false;
169             }
170         }
```

The comment says that it returns true if the typist's progress has reached or passed the passage length, but the condition only checks if they have reached the passage length, not passed.

Therefore, I have modified the code:

```
167     /**
168      * Returns true if the given typist has completed the full passage.
169      *
170      * @param theTypist the typist to check
171      * @return true if their progress has reached or passed the passage length
172      */
173     private boolean raceFinishedBy(Typist theTypist)
174     {
175         // Ty was confident this condition was correct
176         if (theTypist.getProgress() >= passageLength)
177         {
178             return true;
179         }
180         else
181         {
182             return false;
183         }
184     }
185
```

```
/**
 * Prints the current state of the race to the terminal.
 * Shows each typist's position along the passage, burnout state,
 * and a WPM estimate based on current progress.
 */
private void printRace()
{
    System.out.print('\u000C'); // Clear terminal

    System.out.println("  TYPING RACE — passage length: " + passageLength + " chars");
    multiplePrint('=', passageLength + 3);
    System.out.println();

    printSeat(seat1Typist);
    System.out.println();

    printSeat(seat2Typist);
    System.out.println();

    printSeat(seat3Typist);
    System.out.println();

    multiplePrint('=', passageLength + 3);
    System.out.println();
    System.out.println("  [zz] = burnt out    [<] = just mistyped");
}
```

In the screenshot shown in the specification, the burnt out logo is a tilde.

Also, in the screenshot, the number of equals is 2 more than the passage length, not 3.

My code:

```
/**
 * Prints the current state of the race to the terminal.
 * Shows each typist's position along the passage, burnout state,
 * and a WPM estimate based on current progress.
 */
private void printRace()
{
    System.out.print('\u000C'); // Clear terminal

    System.out.println("  TYPING RACE — passage length: " + passageLength + " chars");
    multiplePrint(aChar: '=', passageLength + 2);
    System.out.println();

    printSeat(seat1Typist);
    System.out.println();

    printSeat(seat2Typist);
    System.out.println();

    printSeat(seat3Typist);
    System.out.println();

    multiplePrint(aChar: '=', passageLength + 2);
    System.out.println();
    System.out.println("  [~] = burnt out    [<] = just mistyped");
}
```

```
/**
 * Prints a single typist's lane.
 *
 * Examples:
 *   |          ⌨           | TURBOFINGERS (Accuracy: 0.85)
 *   |    [zz]              | HUNT_N_PECK  (Accuracy: 0.40) BURNT OUT (2 turns)
 *
 * Note: Ty forgot to show when a typist has just mistyped. That would
 * be a nice improvement — perhaps a [<] marker after their symbol.
 *
 * @param theTypist the typist whose lane to print
 */
private void printSeat(Typist theTypist)
{
    int spacesBefore = theTypist.getProgress();
    int spacesAfter  = passageLength - theTypist.getProgress();

    System.out.print('|');
    multiplePrint(' ', spacesBefore);

    // Always show the typist's symbol so they can be identified on screen.
    // Append ~ when burnt out so the state is visible without hiding identity.
    System.out.print(theTypist.getSymbol());
    if (theTypist.isBurntOut())
    {
        System.out.print('~');
        spacesAfter--; // symbol + ~ together take two characters
    }
```

Firstly, the comment says that the symbol + ~ together take 2 characters but the code subtracts only 1 from spacesAfter. So I modified the comment.

Currently there is no way to check if the typist has just mistyped, so I need to add that.

I added a new attribute to the Typist class – justMistyped:

```
21      private String name;
22      private char symbol;
23      private int progress;
24      private boolean burntOut;
25      private int turnsRemaining;
26      private double accuracy;
27      private boolean justMistyped;
```

I added a hasJustMistyped method:

```
129
130     /**
131      * Returns if the typist has just mistyped
132      * This is calculated if they have called the 'slideBack' method
133      * and have not yet called the 'advanceTypist' method
134      *
135      * @return true if they have just mistyped
136      */
137     public boolean hasJustMistyped() {
138         return this.justMistyped;
139     }
```

This returns the status of whether the typist has just mistyped.

However, I will need to also reset this value when the typists are reset:

```
141     /**
142      * Resets the typist to their initial state, ready for a new race.
143      * Progress returns to zero, burnout is cleared entirely.
144      */
145     public void resetToStart()
146     {
147         this.progress = 0;
148         this.burntOut = false;
149         this.turnsRemaining = 0;
150         this.justMistyped = false;
151     }
```

If the user mistypes (so if the slideBack method is called) then ‘justMistyped’ should be set to true:

```java
    /**
     * Moves the typist backwards by a given number of characters (a mistype).
     * Progress cannot go below zero – the typist cannot slide off the start.
     *
     * @param amount the number of characters to slide back (must be positive)
     */
    public void slideBack(int amount)
    {
        this.justMistyped = true;
        int newProgress = this.progress - amount;
        if (newProgress < 0) { // if negative
            this.progress = 0; // then put progress equal to zero
        }
        else { // if not negative
            this.progress = newProgress;
        }
    }
```

However, when they type normally again (when the advanceTypist method is called), then the value should be reset (set to false). Since advanceTypist is in the TypingRace class, it cannot access the justMistyped attribute directly, therefore I will need to create a method to reset the value to false.

```java
    /**
     * Resets the attribute 'justMistyped' (i.e. sets it to false)
     *
     * */
    public void resetJustMistyped() {
        this.justMistyped = false;
    }
```

```
121     /**
122      * Simulates one turn for a typist.
123      *
124      * If the typist is burnt out, they recover one turn's worth and skip typing.
125      * Otherwise:
126      *   - They may type a character (advancing progress) based on their accuracy.
127      *   - They may mistype (sliding back) — the chance of a mistype should decrease
128      *     for more accurate typists.
129      *   - They may burn out — more likely for very high-accuracy typists
130      *     who are pushing themselves too hard.
131      *
132      * @param theTypist the typist to advance
133      */
134     private void advanceTypist(Typist theTypist)
135     {
136         if (theTypist.isBurntOut())
137         {
138             // Recovering from burnout — skip this turn
139             theTypist.recoverFromBurnout();
140             return;
141         }
142 
143         if (theTypist.hasJustMistyped()) {
144             theTypist.resetJustMistyped();
145         }
```

Now that I have made it clear if the typist has just mistyped, I need to set the symbol that they just mistyped, if they mistype.

```java
    /**
     * Prints a single typist's lane.
     *
     * Examples:
     *   |           ⌨           | TURBOFINGERS (Accuracy: 0.85)
     *   |           P  [<]      | QWERTY_QUEEN (Accuracy: 0.60) ← just mistyped
     *   |    [zz]               | HUNT_N_PECK  (Accuracy: 0.40) BURNT OUT (2 turns)
     *
     * Note: Ty forgot to show when a typist has just mistyped. That would
     * be a nice improvement — perhaps a [<] marker after their symbol.
     *
     * @param theTypist the typist whose lane to print
     */
    private void printSeat(Typist theTypist)
    {
        int spacesBefore = theTypist.getProgress();
        int spacesAfter  = passageLength - theTypist.getProgress();

        System.out.print('|');
        multiplePrint(aChar: ' ', spacesBefore);

        // Always show the typist's symbol so they can be identified on screen.
        // Append ~ when burnt out so the state is visible without hiding identity.
        System.out.print(theTypist.getSymbol());
        if (theTypist.isBurntOut())
        {
            System.out.print('~');
            spacesAfter--; // ~ takes one character
        }
        else if (theTypist.hasJustMistyped()) {
            System.out.print("  [<]");
            spacesAfter = spacesAfter - 5; // 2 spaces + 2 square brackets + < are 5 in total
        }
```

```java
        else if (theTypist.hasJustMistyped()) {
            System.out.print("  [<]");
            spacesAfter = spacesAfter - 6; // symbol + 2 spaces + 2 square brackets + < are 6 in total
        }

        multiplePrint(aChar: ' ', spacesAfter);
        System.out.print('|');
        System.out.print(' ');

        // Print name and accuracy
        if (theTypist.isBurntOut())
        {
            System.out.print(theTypist.getName()
                + " (Accuracy: " + theTypist.getAccuracy() + ")"
                + " BURNT OUT (" + theTypist.getBurnoutTurnsRemaining() + " turns)");
        }
        else if (theTypist.hasJustMistyped()) {
            System.out.print(theTypist.getName()
                + " (Accuracy: " + theTypist.getAccuracy() + ")"
                + " ← just mistyped");
        }
        else
        {
            System.out.print(theTypist.getName()
                + " (Accuracy: " + theTypist.getAccuracy() + ")");
        }
    }
```

I implemented the performance adjustment – where a typist burning out slightly decreases their accuracy, and a typist winning a race slightly increases their accuracy. However, the program represents the accuracy to a very high number of decimal places:

```
  TYPING RACE — passage length: 68 chars
========================================================================
|                                                                      ①| TURBOFINGERS (Accuracy: 0.6899999999999998)
| ②  [<]                                                                | QWERTY_QUEEN (Accuracy: 0.5199999999999999) ← just mistyped
|③  [<]                                                                 | HUNT_N_PECK (Accuracy: 0.25999999999999995) ← just mistyped
========================================================================
  [~] = burnt out    [<] = just mistyped
And the winner is... TURBOFINGERS
Final accuracy: 0.7099999999999999 (improved from 0.6899999999999998)
```

So I have decided to use the DecimalFormat subclass to format the accuracy to 2 decimal places.

I imported the subclass at the top of TypingRace.java:

```
3   import java.text.DecimalFormat;
```

And I created a new object of it at the beginning of the class so that it can be used throughout without having to define it everytime:

```
16  public class TypingRace
17  {
18      private int passageLength;   // Total characters in the passage to type
19      private Typist seat1Typist;
20      private Typist seat2Typist;
21      private Typist seat3Typist;
22
23      // Accuracy thresholds for mistype and burnout events
24      // (Ty tuned these values "by feel". They may need adjustment.)
25      private static final double MISTYPE_BASE_CHANCE = 0.8;
26      private static final int    SLIDE_BACK_AMOUNT   = 2;
27      private static final int    BURNOUT_DURATION    = 2;
28      private DecimalFormat df = new DecimalFormat("0.00");
29
```

I then use it when printing the accuracy:

```
117     for (Typist t : players) {
118         if (raceFinishedBy(t)) {
119             double oldAcc = t.getAccuracy(); // save old accuracy
120             t.setAccuracy(oldAcc + 0.02); // slightly increase winner's accuracy
121
122             System.out.println("And the winner is... "+t.getName());
123             System.out.println("Final accuracy: "+df.format(t.getAccuracy())+" (improved from "+df.format(oldAcc)
124         }
125     }
126 }
127
```

But I do not round for calculations as this could lead to inaccurate calculations.

If two typists finish on the same turn, the program puts both as the winner.

Also, SLIDE_BACK_AMOUNT and BURNOUT_DURATION are final variables so cannot be modified for extra features in the future.

After running the simulation, I discovered that a mistype base chance of 0.8 was extremely slow in reality, therefore I have lowered it to 0.3.